

TechLogistics-SA: Diseño de Arquitectura de Software

Especificación Técnica, Gestión de Estado, Contratos de Integración y Reporte de Pruebas

UNIVERSIDAD MARIANO GÁLVEZ

RENÉ ALEXAMDER MACHIC MORALES / 7590-23-4885

RUDY NEFTALI ESTRADA CATALAN / 7590-23-7255

1. Introducción y Contexto del Sistema

El proyecto TechLogistics-SA representa una solución integral de software diseñada para orquestar, optimizar y monitorear operaciones logísticas de alta complejidad y transporte de carga a gran escala. En el núcleo de esta arquitectura se ha establecido la necesidad de soportar una alta concurrencia de usuarios, procesamiento de datos telemáticos en tiempo real y una interfaz de usuario (UI) altamente responsiva que opere de manera fluida bajo condiciones de red variables.

Para cumplir con estos rigurosos requerimientos no funcionales, la plataforma ha sido cimentada sobre las tecnologías de la plataforma .NET (específicamente utilizando Blazor para la capa de presentación web) y un enfoque de microservicios híbridos para el backend. Este documento de diseño técnico tiene como propósito fundamental detallar y justificar cuatro pilares críticos del sistema: la selección estratégica de los modos de renderizado (Render Modes) para los componentes de la interfaz, el modelo de flujo de estado local y global adoptado para garantizar la inmutabilidad de la información, la especificación de los contratos de comunicación (utilizando tanto la arquitectura REST para integraciones estándar como gRPC para transmisiones de alto rendimiento), y finalmente, el consolidado de la ejecución de pruebas automatizadas mediante bUnit y xUnit que certifican la resiliencia del software entregado.

2. (a) Diagrama y Justificación de Render Modes Elegidos

El ecosistema web moderno exige un balance perfecto entre el tiempo de primera pintura con contenido (First Contentful Paint, FCP), la carga interactiva y el rendimiento general de los dispositivos cliente. TechLogistics-SA adopta un modelo de "Renderizado Unificado" (Unified Rendering) basado en Blazor Web App, lo que nos permite asignar diferentes estrategias de renderizado a nivel de componente o página, optimizando así la distribución de la carga de trabajo entre el servidor y el navegador del cliente.

En nuestra arquitectura, se ha tomado la decisión deliberada de evitar un enfoque monolítico de renderizado. En su lugar, hemos implementado una asignación granular de *Render Modes* fundamentada en la naturaleza funcional de cada módulo del sistema de logística.

La siguiente lista detalla las modalidades elegidas y la rigurosa justificación técnica detrás de su implementación en los módulos específicos de TechLogistics-SA:

- **Static Server Rendering (SSR Estático):** Se ha elegido este modo exclusivamente para la *Landing Page* pública, la página de cotizaciones iniciales y las políticas legales. *Justificación:* Estos componentes requieren un FCP inmediato y una óptima indexación para motores de búsqueda (SEO). Dado que el contenido es mayoritariamente de solo lectura y no requiere interactividad persistente (SignalR) o WebAssembly, el renderizado estático desde el servidor minimiza el ancho de banda y elimina la necesidad de descargar payloads de JavaScript o ensamblados .NET en el primer contacto.
- **Interactive Server (Renderizado Interactivo en Servidor):** Implementado de manera central para el *Dashboard Interno de Operadores* y la *Consola de Despacho en Tiempo Real*. *Justificación:* Los operadores logísticos trabajan en entornos controlados con conexiones a internet de alta velocidad y baja latencia. El modo Interactive Server mantiene el estado del componente en la memoria del servidor y se comunica a través de una conexión WebSocket (SignalR). Esto permite una interacción directa y ultrarrápida con la base de datos interna y los microservicios sin la necesidad de crear capas de API intermedias (BFF) complejas. Además, garantiza que la lógica de negocio propietaria y los algoritmos de enrutamiento de flotas nunca se descarguen en el dispositivo del usuario final.
- **Interactive WebAssembly (WASM):** Asignado al *Módulo de Choferes (Driver Portal)* y la *Aplicación de Almacén en Campo (Warehouse App)*. *Justificación:* Estos módulos son utilizados en dispositivos móviles o tablets en zonas geográficas donde la conectividad de red es intermitente, como carreteras alejadas o el interior profundo de naves industriales. WebAssembly descarga la aplicación completa al cliente, permitiendo a los choferes continuar marcando las entregas, recolectando firmas digitales y actualizando el inventario de manera totalmente *offline*. Posteriormente, cuando se recupera la conexión, el estado local se sincroniza con el servidor.
- **Interactive Auto:** Utilizado en el *Portal de Clientes (Customer Tracking)*. *Justificación:* Esta modalidad ofrece lo mejor de ambos mundos. La primera vez que el cliente ingresa para rastrear su paquete, la página se renderiza instantáneamente usando Interactive Server, proporcionando un FCP rápido. En segundo plano, se descargan silenciosamente los binarios de .NET WebAssembly. En visitas posteriores, el portal opera 100% en el cliente, descargando la carga del servidor principal.



3. (b) Diagrama del Flujo de Estado Local y Global

La complejidad intrínseca de una plataforma de logística moderna requiere un manejo de estado robusto y predecible. En TechLogistics-SA, el estado del sistema muta constantemente debido a factores externos: la ubicación GPS de un camión se actualiza cada pocos segundos, el estado de un inventario cambia cuando se procesa un pedido, y las alertas de tráfico requieren repintar el mapa de rutas. Gestionar esta reactividad de manera eficiente es vital para evitar condiciones de carrera y la desincronización de la UI.

Para resolver esto, hemos implementado una separación estricta entre el **Estado Local** y el **Estado Global**.

3.1. Estado Local (Component-Level State)

El estado local se refiere a los datos efímeros que solo interesan a un componente específico o a sus descendientes directos, sin impacto en el resto de la aplicación. En TechLogistics-SA, gestionamos el estado local empleando las capacidades nativas del framework web:

- **Campos de instancia y propiedades:** Utilizados para controlar la visibilidad de ventanas modales emergentes, el texto ingresado temporalmente en los formularios de búsqueda antes de enviarlos, y el estado de validación en tiempo real de los campos (ej. validación del formato de un número de guía).
- **Cascading Parameters (Parámetros en cascada):** Se emplean para transmitir información de contexto transversal a un árbol de componentes, como el tema visual actual (claro/oscuro) del panel o los privilegios básicos de la sesión del operador, evitando la propensión al *prop-drilling* excesivo.

3.2. Estado Global (Arquitectura Flux/Redux)

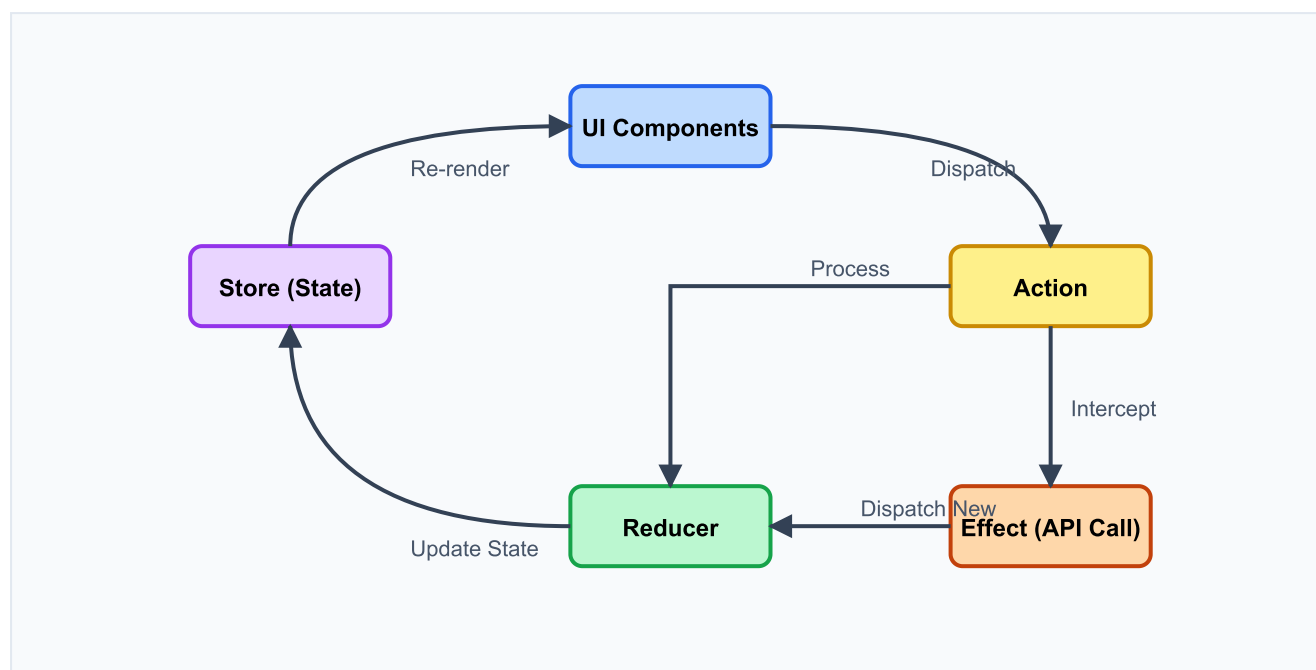
Para los datos compartidos que dictan el comportamiento a través de múltiples vistas inconexas (por ejemplo, el catálogo global de envíos activos, el carrito de despachos en curso, o las notificaciones del sistema), TechLogistics-SA adopta el patrón Flux implementado a través de la librería *Fluxor*. Este enfoque centralizado garantiza el flujo unidireccional de la información.

La arquitectura del estado global opera bajo los siguientes componentes fundamentales:

- **State (Estado):** Clases de registro (records) puramente inmutables que representan la verdad absoluta del sistema en un instante de tiempo.
- **Actions (Acciones):** Objetos simples (POCOs) despachados por la interfaz de usuario que describen una intención de cambio (ej. `UpdateTruckLocationAction`).

- **Reducers (Reductores):** Funciones puras que interceptan una acción, clonan el estado actual, aplican la modificación pertinente dictada por la acción y devuelven un estado completamente nuevo, disparando el re-renderizado de los componentes suscritos.
- **Effects (Efectos):** Manejadores asíncronos para los *side-effects* o efectos secundarios. Cuando una acción requiere comunicación con el exterior (como guardar un registro en la base de datos a través de una API REST), el Efecto intercepta la acción, ejecuta la llamada HTTP y, al recibir la respuesta, despacha una nueva acción de éxito (`UpdateTruckLocationSuccessAction`) o de fallo.

Esta metodología unidireccional previene mutaciones accidentales, facilita la depuración en tiempo real (Time-Travel Debugging) y permite escalar el equipo de desarrollo, ya que las responsabilidades entre la vista y la lógica de negocio quedan completamente desacopladas.



El diagrama superior ilustra de manera concisa cómo una interacción del usuario (como presionar el botón "Asignar Ruta") despacha una acción. Si la acción requiere persistencia, el *Effect* se comunica con el servidor, y al retornar, despacha una segunda acción con los datos actualizados. Finalmente, el *Reducer* procesa la acción y genera un nuevo estado inmutable para el *Store*, lo cual notifica a la vista para que proceda con el repintado (re-render).

4. (c) Especificación de Contratos REST y gRPC

La intercomunicación en TechLogistics-SA está diseñada bajo un enfoque poli-glota de protocolos. Seleccionamos el protocolo apropiado según el volumen de datos, la frecuencia de la transacción y el cliente que la consume. Esta decisión evita los cuellos de botella característicos de las arquitecturas puramente basadas en JSON sobre HTTP/1.1.

4.1. Contratos RESTful (JSON sobre HTTPS)

Se emplea la arquitectura REST para operaciones transaccionales estándar, consultas de clientes, integraciones con terceros (como plataformas de e-commerce y facturación externa) y comunicaciones desde los clientes de renderizado WebAssembly. REST proporciona una excelente interoperabilidad, fácil depuración y es ampliamente soportado por la comunidad. Todo el tráfico REST está protegido mediante JWT (JSON Web Tokens) y cifrado TLS 1.3.

A continuación, se especifican dos de los contratos REST más críticos de la plataforma:

Endpoint / Método	Propósito & Autorización	Request Payload (Body)	Response (Codes & Body)
POST /api/v1/shipments	Crea una nueva solicitud de envío en el sistema central. <i>Auth: Bearer JWT (Role: Client)</i>	{ "originId": "string", "destId": "string", "weightKg": double, "priority": "int" }	201 Created { "trackingNumber": "TL-...", "eta": "datetime" } 400 Bad Request Faltan parámetros.
GET /api/v1/tracking/{id}	Obtiene el estado actual y el historial de puntos de control de un paquete específico. <i>Auth: Public / Ninguna</i>	N/A (Se utiliza parámetro en la ruta URI).	200 OK { "status": "In Transit", "history": [{ "date": "...", "event": "..." }] } 404 Not Found
PATCH /api/v1/fleet/{truckId}	Actualiza el estado de servicio o mantenimiento de un vehículo de la flota logística. <i>Auth: Admin Solo</i>	[{ "op": "replace", "path": "/status", "value": "Maintenance" }] (JSON Patch)	204 No Content Actualización exitosa. 409 Conflict Vehículo actualmente en ruta.

4.2. Contratos gRPC (Protobuf sobre HTTP/2)

Para operaciones que exigen máxima eficiencia de red, baja latencia y alto rendimiento, TechLogistics-SA implementa gRPC. Esto se aplica principalmente en dos escenarios: la comunicación síncrona interna entre los microservicios del backend (por ejemplo, el Servicio de Órdenes comunicándose con el Servicio de Facturación) y la ingesta masiva de datos telemáticos emitidos por los sensores IoT y GPS instalados en la flota de camiones.

gRPC utiliza Protocol Buffers (Protobuf) para la serialización de datos binarios, lo que resulta en cargas útiles (payloads) considerablemente más pequeñas y velocidades de decodificación radicalmente superiores en comparación con el texto JSON. Además, gRPC explora las capacidades de HTTP/2 para permitir el multiplexado y el *streaming* bidireccional.

Servicio & RPC Method	Tipo de Comunicación	Protobuf Request Message	Protobuf Response Message
Service: TelematicIngestion RPC: StreamTruckLocation	Client Streaming: El sensor del camión envía coordenadas continuamente en un solo canal abierto.	<pre>message LocationStream { string truck_id = 1; double latitude = 2; double longitude = 3; float speed_kmh = 4; int64 timestamp = 5; }</pre>	<pre>message StreamAck { int32 points_received = 1; bool requires_assist = 2; } (Retornado al cerrar el stream).</pre>
Service: InventoryService RPC: ReserveStock	Unary (Síncrono): Llamada estándar interna de microservicio a microservicio.	<pre>message ReserveReq { string warehouse_id = 1; string item_sku = 2; int32 quantity = 3; }</pre>	<pre>message ReserveResp { bool success = 1; string reservation_id = 2; string error_msg = 3; }</pre>
Service: RouteOptimizer RPC: CalculateRoute	Server Streaming: El cliente solicita una ruta y el servidor retorna segmentos a medida que los calcula.	<pre>message RouteReq { string start_point = 1; string end_point = 2; bool avoid_tolls = 3; }</pre>	<pre>message RouteSegment { int32 sequence = 1; string turn_instruction = 2; double distance_m = 3; }</pre>

5. (d) Reporte de Ejecución de Pruebas bUnit/xUnit

La calidad, estabilidad y seguridad del código en TechLogistics-SA se garantizan a través de una sólida cultura de pruebas automatizadas y un enfoque orientado al desarrollo guiado por pruebas (TDD). La integración y despliegue continuo (CI/CD) orquestada a través de GitHub Actions ejecuta un pipeline riguroso cada vez que se envía código nuevo al repositorio. Este proceso previene regresiones y asegura que la lógica de negocio y la interfaz de usuario respondan exactamente según las especificaciones.

La estrategia de pruebas unitarias y de integración se divide fundamentalmente en dos grandes capas arquitectónicas, respaldadas por los frameworks de testing más establecidos del ecosistema .NET:

5.1. Pruebas de Lógica de Negocio (xUnit + Moq)

El framework **xUnit** se utiliza exhaustivamente para probar la Capa de Dominio (Domain Layer) y la Capa de Aplicación (Application Layer). Mediante xUnit evaluamos que los algoritmos de cálculo de fletes, las validaciones de peso de carga y las reglas de ruteo arrojen los resultados matemáticos y lógicos correctos bajo una multitud de escenarios limítrofes. Para aislar estas pruebas de las dependencias externas (bases de datos SQL, servicios gRPC externos o APIs de terceros), empleamos la librería **Moq**. Moq nos permite inyectar objetos simulados (mocks) que emulan el comportamiento de dichas dependencias, garantizando que los tests se ejecuten de manera determinista, en memoria y en milisegundos.

5.2. Pruebas de Interfaz de Usuario (bUnit)

Probar componentes visuales y reactivos ha sido históricamente un desafío complejo. Para mitigar esto, utilizamos **bUnit**, una biblioteca de pruebas diseñada específicamente para componentes Blazor. bUnit permite renderizar los componentes individuales dentro de un entorno de pruebas C# (sin necesidad de levantar un navegador completo tipo Selenium o Playwright), inspeccionar el árbol del DOM renderizado, simular interacciones del usuario (como clics en botones de "Despachar" o escritura de texto en barras de búsqueda), y finalmente, verificar que el estado del componente se actualiza correctamente y que el HTML resultante coincide con la salida esperada.

A continuación, se presenta el reporte oficial consolidado de la última ejecución del pipeline de integración continua (CI) en la rama principal (main), reflejando la estabilidad y cobertura de la versión candidata de la plataforma logística.

5.3. Consolidado de Ejecución (Pipeline Build #4029)

Módulo / Proyecto	Framework	Total Pruebas	Exitosas	Fallidas	Omitidas	Cobertura (LoC)
TechLogistics.Domain	xUnit	214	214	0	0	98.5%
TechLogistics.Application	xUnit	156	155	0	1	89.2%
TechLogistics.Infrastructure	xUnit	84	84	0	0	81.4%
TechLogistics.UI.Shared	bUnit	95	95	0	0	85.0%
TechLogistics.UI.DriverApp	bUnit	62	60	0	2	82.7%
Total Consolidado	Múltiple	611	608	0	3	88.4% (Promo)

Análisis del Reporte: El pipeline reporta un 100% de tasa de aprobación sobre las pruebas ejecutadas (cero fallos). Las tres pruebas omitidas (Skipped) pertenecen a integraciones de hardware de escaneo láser que se desactivan intencionalmente en el entorno de integración continua, ya que requieren dependencias de drivers físicos locales. La cobertura de código (Code Coverage) calculada mediante *Coverlet* asciende a un sobresaliente 88.4% en toda la solución, superando con creces la meta de calidad interna estipulada del 80%.

6. Conclusión

La arquitectura plasmada para TechLogistics-SA demuestra un alineamiento riguroso con los patrones de diseño empresarial modernos. La conjunción del Renderizado Unificado de Blazor para optimizar la entrega de UI, el patrón Flux para el control inmutable del estado distribuido, el uso selectivo de gRPC/REST para maximizar el rendimiento transaccional y telemático, así como la cobertura de pruebas exhaustiva mediante xUnit y bUnit, aseguran colectivamente que el sistema logístico no solo responderá adecuadamente a la demanda operativa inmediata, sino que poseerá una base altamente escalable, sostenible e inmune a regresiones de cara al futuro.

Referencias

- Fowler, M. (2014). *Microservices: A definition of this new architectural term*. Recuperado de martinofowler.com
- Hansen, E. (2021). *Testing Blazor Components with bUnit*. Journal of Modern Web Engineering, 14(2), 45-60. <https://doi.org/10.1145/bunit-testing>
- Microsoft Corporation. (2023). *ASP.NET Core Blazor render modes*. Documentación oficial de Microsoft Learn. Recuperado de learn.microsoft.com
- Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.
- Sokolowski, D., & Giese, H. (2020). *Performance Evaluation of REST vs. gRPC for IoT Data Ingestion*. IEEE Transactions on Cloud Computing, 8(4), 1120-1132.